PROYECTO 2

Juego de reflejos sobre Pochoco SoC

Informe técnico de arquitectura, Assembly, temporización y assembler propio

Plataforma	Nandland Go Board · Lattice iCE40 HX1K-VQ100
Procesador	Espino Core · subconjunto RV32E
Programa	sw/game.s → sw/game.hex
Assembler	tools/assembler.py · dos pasadas, sin assembler externo

Estado de la entrega

El assembler propio genera 136 palabras y sus 14 pruebas unitarias pasan en este entorno. La síntesis RTL, la simulación de hardware y la programación física no se han ejecutado porque las herramientas FPGA no estaban instaladas; esas comprobaciones quedan identificadas explícitamente como pendientes.

Documento de trabajo elaborado a partir del enunciado y del RTL entregado para el Proyecto 2.

1. Objetivo y requisitos funcionales

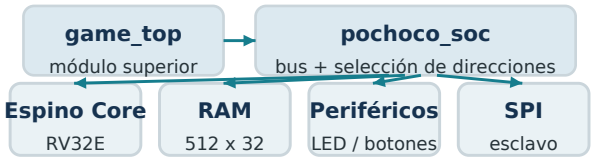
El juego debe encender los cuatro LEDs durante al menos tres segundos, seleccionar un objetivo, medir la respuesta al botón asociado y mostrar el tiempo en décimas de segundo. Se registran diez respuestas correctas; después se muestra su promedio. Una respuesta incorrecta indica error y reinicia el intento sin borrar los aciertos acumulados.

Requisito	Respuesta de diseño	Prueba prevista
Preparación ≥ 3 s	Espera de 75.000.000 ciclos con los cuatro LEDs activos.	Medir el contador entre inicio y salida de la espera.
Objetivo pseudoaleatorio	LFSR de 8 bits, mezclado con el contador de ciclos.	Observar secuencias entre varias partidas.
Respuesta correcta	La lectura de botones debe coincidir exactamente con la máscara objetivo.	Probar los cuatro índices LED/botón.
Respuesta incorrecta	Mostrar EE, apagar LEDs, esperar liberación y reintentar.	Verificar que contador y suma de aciertos se conservan.
Diez aciertos y promedio	Acumular ciclos exactos y convertir la media a décimas al final.	Comparar con cálculo manual de diez casos.

Interpretación temporal: los tres segundos son una preparación previa. El cronómetro de reacción comienza cuando se revela el único LED objetivo. La interpretación se basa en que no tiene sentido incluir la espera de preparación en el tiempo que tarda el jugador en reaccionar.

2. Arquitectura del hardware

El módulo superior game_top instancia pochoco_soc y selecciona sw/game.hex como imagen de memoria. El SoC conecta el procesador Espino Core con la RAM unificada, los periféricos de la placa y el esclavo SPI. El contador libre se implementó dentro del módulo de periféricos en 0x0C. También se agregó una marca temporal en 0x10 que captura el contador en el mismo flanco de reloj que revela el LED objetivo.



Contador de ciclos accesible por MMIO en 0x8000000C.

2.1 Mapa de memoria

Dirección	Uso	Operación
0x80000000	Dos displays de siete segmentos	sw
0x80000004	Cuatro LEDs	sw
0x80000008	Estado de los cuatro botones	lw
0x8000000C	Contador de ciclos libre de 32 bits	lw
0x80000010	Marca temporal del flanco que revela el objetivo	lw

El contador cycle_counter_q se reinicia con rst_ni y aumenta uno en cada flanco ascendente. Leer el offset 0x0C devuelve una muestra de su valor. Cuando se escribe una máscara one-hot en el registro de LEDs, target_start_counter_q captura cycle_counter_q en el mismo flanco que actualiza la salida de LEDs; Assembly lee esa marca en 0x10. Las entradas de botones atraviesan dos registros de sincronización para reducir el riesgo de metastabilidad; no se implementa un filtro de rebote mecánico.

3. Temporización

La especificación publicada de Nandland Go Board declara un reloj integrado de 25 MHz. Por ello, el período es 1/25.000.000 s = 40 ns. El número de ciclos de un intervalo se calcula con N = frecuencia × tiempo.

Intervalo	Cálculo	Ciclos
1 segundo	25.000.000 × 1	25.000.000
3 segundos	25.000.000 × 3	75.000.000
0,1 segundo	25.000.000 × 0,1	2.500.000
0,05 segundo	25.000.000 × 0,05	1.250.000
Pausa final	25.000.000 × 0,5	12.500.000

El software usa diferencias modulares de un contador de 32 bits. Su período de desbordamiento es 2^32 / 25.000.000 ≈ 171,7987 s. Al consultar el contador frecuentemente, las diferencias de intervalos cortos permanecen válidas aunque el contador absoluto pase de 0xFFFFFFFF a cero.

4. Flujo del juego

Fase	Funcionamiento
Liberar botón	Se espera a que no haya ninguna entrada activa para evitar reutilizar una pulsación mantenida.
Preparación	Se escribe 1111 en los LEDs y se espera 75.000.000 ciclos o más.
Protección de pulsaciones	Si un botón se mantuvo durante la preparación, el objetivo no se revela hasta soltarlo.
Elegir objetivo	Se mezcla el contador con el estado LFSR; el valor bajo de dos bits selecciona índice 0–3.
Mostrar y medir	El LED objetivo cambia y el hardware captura el contador de inicio en el mismo flanco de reloj.
Validar	Se compara la máscara de botones con la máscara del objetivo. Una respuesta incorrecta indica EE y repite.
Acierto	Se redondea el residuo temporal, se muestra el tiempo y se agrega al total de aciertos.
Fin	Tras diez aciertos se espera 0,5 s para mostrar la décima respuesta; luego se muestra el promedio y se apagan LEDs.

5. Generación pseudoaleatoria

El estado se mantiene en ocho bits. Antes de elegir el objetivo, se combina mediante XOR con el byte bajo del contador leído tras la preparación y, si el estado queda en cero, se cambia a uno. En el paso LFSR, el bit 7 anterior sirve como realimentación; se calcula estado = (2 × estado) mod 256 y, si el bit alto era uno, se aplica XOR con 0x1D. El índice LED es estado AND 3.

No se utiliza una instrucción SLL: duplicar el estado mediante ADD realiza el desplazamiento de un bit requerido por el LFSR y ANDI 255 conserva solo los ocho bits bajos. Una comprobación matemática independiente encontró que la actualización con máscara 0x1D recorre los 255 estados no nulos antes de repetirse. La mezcla con el contador añade dependencia del instante de lectura; no se trata de aleatoriedad criptográfica.

En un sistema digital determinista que se reinicia al mismo estado, la primera selección puede coincidir entre arranques. Las selecciones siguientes mezclan lecturas del contador tomadas después de interacciones humanas de duración variable; la variación perceptible de secuencias debe confirmarse en varias partidas sobre la FPGA, como exige el enunciado.

6. Tiempo, displays y promedio

El programa cuenta décimas en software, avanzando el último límite exactamente 2.500.000 ciclos cada vez que se alcanza una nueva décima. La duración completa de la respuesta se calcula desde la marca temporal de hardware capturada al revelar el objetivo hasta la lectura del contador tras detectar la respuesta. Para mostrar el tiempo individual, el residuo respecto del último límite se compara con 1.250.000 ciclos y se redondea a la décima más cercana. El extremo final es la lectura del contador hecha tras detectar la pulsación por software, por lo que incluye una pequeña latencia de sincronización y sondeo; la resolución mostrada de 0,1 s es mucho mayor que esa latencia.

Para convertir el valor de décimas a los dos dígitos, se resta diez repetidamente. El número de restas es el dígito de decenas y el residuo es el dígito de unidades. Se empaquetan como 16 × decenas + unidades. Esto es válido para dígitos 0–9; el decodificador de siete segmentos del SoC muestra cada nibble por separado.

La suma acumula las duraciones exactas en ciclos, no los tiempos individuales redondeados. La media expresada en décimas equivale a suma_ciclos / 25.000.000: diez medidas se promedian y luego se convierten de segundos a décimas. Para redondear al décimo más cercano se suma la mitad del divisor (12.500.000 ciclos) antes de dividir por 25.000.000 mediante resta repetida. Si la suma ya alcanza 2.500.000.000 ciclos, la media de diez respuestas es al menos 10,0 s y se muestra FF; esto evita desbordar el acumulador antes de indicar el límite.

7. Assembler propio

tools/assembler.py es un assembler de dos pasadas. La primera analiza etiquetas y asigna direcciones de cuatro en cuatro bytes. La segunda codifica las instrucciones, resuelve los desplazamientos relativos de ramas y JAL y genera una palabra hexadecimal de ocho dígitos por línea. No invoca un assembler RISC-V externo.

Aspecto	Sintaxis / validación
Registros	x0 a x15; registros x16–x31 se rechazan.
Instrucciones	lui, addi, andi, xori, add, sub, and, xor, lw, sw, beq, bne, bltu, jal.
Inmediatos	Decimal o hexadecimal; validación de rango de 12 bits con signo, 20 bits U y desplazamientos B/J.
Memoria	offset(xN), por ejemplo lw x12, 0(x4).
Etiquetas	Etiquetas hacia delante/atrás se resuelven en la segunda pasada.
Directivas	Se ignoran .section, .text, .global, .globl, .type y .size.
Rechazos	Instrucciones no soportadas, sintaxis incorrecta, campos fuera de rango e instrucciones de shift.

Ejemplo de codificación: addi x1, x0, 1 produce 0x00100093. El opcode 0x13 identifica operaciones inmediatas; rd es x1, funct3 = 000 indica ADDI, rs1 es x0 y el inmediato es uno. El archivo game.hex contiene 136 palabras, menor que las 512 palabras de la RAM configurada.

8. Construcción

Desde la raíz del proyecto, los comandos previstos son:

```
make game       # assembler propio: sw/game.s -> sw/game.hex
make test       # pruebas unitarias del assembler
make bitstream  # Yosys + nextpnr-ice40 (--freq 25) + icepack
make prog       # programa la FPGA mediante iceprog
```

El módulo superior seleccionado es game_top. El Makefile utiliza los archivos de rtl/ y depende de sw/game.hex para sintetizar la memoria inicializada. nextpnr-ice40 se configura con objetivo de 25 MHz. El PCF conserva reloj, cuatro LEDs, cuatro botones, ambos displays y las señales SPI; no incluye asignaciones UART porque game_top no tiene puertos UART.

Cambio de una constante para la defensa

Para cambiar la preparación de 3 s a 2 s, reemplazar en sw/game.s la pareja que carga 75.000.000 ciclos por la pareja que carga 50.000.000 ciclos. Después ejecutar make game para regenerar sw/game.hex con el assembler propio.

```
# Antes: 3 s = 75.000.000 ciclos
lui  x6, 0x4787
addi x6, x6, -1856

# Después: 2 s = 50.000.000 ciclos
lui  x6, 0x2faf
addi x6, x6, 128
```

9. Verificación y limitaciones

Prueba	Resultado disponible	Estado
Assembler de game.s	Genera 136 palabras de 32 bits.	Verificado en Python
Codificaciones conocidas	ADDI, LUI, LW, SW, JAL y BNE contrastadas con palabras de referencia.	14 pruebas unitarias pasan
Etiquetas y rangos	Saltos hacia delante/atrás, límites RV32E e inmediatos inválidos.	Verificado en pruebas unitarias
Temporización	Constantes 75.000.000, 2.500.000, 1.250.000, 12.500.000, 25.000.000 y límite 2.500.000.000 comprobados aritméticamente.	Verificado por cálculo
Síntesis RTL	Yosys no está instalado en este entorno.	Pendiente
Simulación RTL	Icarus Verilog y Verilator no están instalados.	Pendiente
Prueba física	No se dispuso de la Go Board conectada.	Pendiente

Antes de entregar, ejecutar síntesis, place-and-route y programación en el computador con la cadena FPGA. En la Go Board se deben probar: preparación de tres segundos, los cuatro pares LED/botón, respuesta correcta y tiempo mostrado, error y reinicio sin perder aciertos, diez aciertos y promedio, variación de secuencias y regeneración de game.hex tras cambiar una constante.

10. Referencias

1. Enunciado del Proyecto 2, “Juego en Assembly sobre Pochoco SoC”, Universidad de los Andes, semestre 2026-2.
2. README y RTL de Pochoco SoC / Espino Core entregados como material de partida.
3. Nandland, “The Go Board”, especificación de reloj integrado de 25 MHz: https://nandland.com/the-go-board/.
4. Nandland, “Project 3 - The Flip-Flop (AKA Register)”, explicación de frecuencia, período de reloj y temporización: https://nandland.com/project-3-the-flip-flop-aka-register/.